2026학년도 졸업 프로젝트 결과보고서

HASH(Rocksdb)

HashSkipList Flush/점/범위 성능 개선 및 멀티스레드 지원

팀명	**돌디비**
소속	**단국대학교 소프트웨어학과**
이름	**신승호**
학번	32212326

목차

1. 프로젝트 개요 2
1.1 소개 2
1.2 개발 동기 2
1.3 기대 효과 3
2. 팀 구성 4
3. 개발 일정 5
4. AI 사용 범위 6
5. 벤치마크 환경 7
5.1 하드웨어와 소프트웨어 7
5.2 측정 방법 7
6. 벤치마크 웹 서버 아키텍처 8
6.1 기술 스택 9
6.2 지표 수집: Prometheus 형식 pull 9
6.3 화면 10
7. 워크로드 설명 12
8. 실제 실험 결과 14
8.1 Flush 시간 14
8.2 멀티 스레드 쓰기 14
8.3 Range query (범위 조회) 15
8.4 Point query (점 조회) 16
8.5 정확도 검증 17
8.6 목표 달성 17

1. 프로젝트 개요

1.1 소개

항목	내용
프로젝트명	**HASH(Rocksdb)**
한 줄 소개	RocksDB의 HashSkipList Memtable을 멀티 스레드와 Range Query까지 지원하도록 다시 설계해, 운영에서 쓸 수 있는 Memtable 선택지로 만든다.
결과물	RocksDB 수정 브랜치 hash_skiplist_improvements(커밋 5개), 성능 측정기 op_trade_bench, 벤치마크 웹 대시보드, 동작 애니메이션 페이지
저장소	github.com/psmsq4/rocksdb_hash_demo (RocksDB 수정본은 submodule rocksdb/ = github.com/psmsq4/rocksdb_hash)

이 보고서는 세 가지 Memtable을 같은 데이터, 같은 조건으로 비교한다.

이름	구현	특징
HEAD	이 프로젝트의 HashSkipListRep	bucket = InlineSkipList, 정렬된 prefix 목록(prefix_list_)과 PrefixOrderedIterator로 복사 없는 key 순서 순회, 동시 쓰기 지원
UPSTREAM	facebook/rocksdb 원래 HashSkipListRep (e591f5f1a)	bucket = SkipList, 동시 쓰기 불가, key 순서 순회 때마다 Memtable 전체 복사
SKIPLIST	RocksDB 기본 Memtable	InlineSkipList 하나에 모든 KV, 동시 쓰기 지원

1.2 개발 동기

HashSkipList는 RocksDB가 기본 제공하는 Memtable 구현 중 하나지만, 다음 세 가지 문제로 실제 운영에서는 거의 쓰이지 않았다.

> 멀티 스레드 쓰기 불가: 여러 스레드가 동시에 써도 RocksDB 쓰기 그룹의 리더 한 스레드가 모두 넣는다.
>
> 느린 Range Query: 종목(prefix)을 넘는 key 순서 조회(total-order)와 flush 때마다 Memtable 전체를 새 SkipList 로 복사한다. 200 만 건이면 iterator 하나를 여는 데 0.3 초가 걸린다.
>
> 비효율적인 메모리 배치: bucket 의 SkipList 가 노드와 KV 를 Arena 블록의 양 끝에서 따로 할당해, entry 하나를 읽을 때 멀리 떨어진 두 곳을 읽는다.

오픈소스 SW 빅데이터 과목에서의 시도와 반성

3학년 1학기 오픈소스SW빅데이터 과목에서 HashSkipList의 Bucket을 SkipList에서 InlineSkipList로 바꾸고, 변경 전보다 10배 이상 빨라졌다는 결과를 발표했다. 그런데 이번에 다시 측정해 보니 그 차이의 대부분은 같은 변경에 섞여 들어간 skiplist 최대 높이 변경(4 → 12)에서 나온 것이었다. 높이를 같게 맞추면 Bucket 교체만의 이득은 워크로드에 따라 약 5~16% 수준이다(무작위 삽입 14%, 큰 데이터의 Get 9~16%).

그래도 그 과목에서 얻은 성과는 분명하다. InlineSkipList는 동시 삽입(InsertConcurrently)을 지원하므로, Bucket 교체를 계기로 HashSkipList가 처음으로 멀티 스레드 쓰기를 지원하게 되었다. 이번 측정에서도 스레드 4개로 쓸 때 HEAD는 UPSTREAM보다 처리량이 1.70배 높다(8장).

과목 발표 때 받은 질문 가운데 "Hash 자료구조를 쓰지 않는 이유는 Range Query 때문"이라는 지적이 남았다. 졸업작품에서는 이 문제를 정면으로 다루기로 하고, Hash bucket 구조를 유지하면서도 key 순서 순회를 복사 없이 할 수 있는 새 Memtable 구조를 구상했다.

	목표
1차 목표	Hash 자료구조의 Range Query 난제 해결: total-order 조회와 flush를 Memtable 복사 없이, 기본 SkipList 수준의 지연으로 처리한다.
2차 목표	HashSkipList가 운영 환경에서 충분히 쓸 수 있는 성능을 갖추고, prefix가 있는 워크로드에서는 기본 SkipList보다 나은 성능을 보여, RocksDB 기반 시스템(Alluxio, MyRocks 등) 개발자가 고려할 만한 Memtable 선택지를 만든다.

해결 방법

멀티 스레드, 메모리 배치: Bucket 을 InlineSkipList 로 교체했다. `Allocate()`가 KV 공간을 InlineSkipList 노드로 받도록 바꿔 노드와 KV 가 한 덩어리가 되었고, bucket 설치는 CAS 로, 동시 삽입은 `InsertConcurrently()`로 처리한다.

Range Query: 입력되는 KV 의 prefix 를 따로 뽑아 InlineSkipList(prefix_list_)로 정렬 상태를 유지한다. 순회할 때는 prefix 경계에서 정렬된 prefix 목록의 다음 prefix 로 넘어가, 그 prefix 의 bucket 으로 바로 이동해 해당 prefix 가 시작하는 지점부터 읽는다(PrefixOrderedIterator).

prefix 등록 비용: 삽입마다 prefix 목록을 찾으면 쓰기가 25~31% 느려졌다. bucket 마다 마지막 prefix 를 캐시(known_prefix)해 새 prefix 일 때만 목록을 찾도록 해 UPSTREAM 과 같은 쓰기 속도를 회복했다.

1.3 기대 효과

Range Query 가 가능한 Hash Memtable: 종목을 넘는 key 순서 조회가 UPSTREAM 대비 약 15,575 배 빨라져 기본 SkipList 와 같은 수준이 된다. iterator 를 열 때 Memtable 크기만큼의 추가 메모리도 쓰지 않는다.

prefix 워크로드에서 기본 SkipList 보다 나은 성능: KV 를 종목별 작은 skiplist 여러 개로 나누는 효과로, 시간순 쓰기는 SkipList 보다 56% 빠르고, 없는 종목 조회는 약 6.4 배 빠르다. flush 도 UPSTREAM 보다 32% 짧다.

멀티 스레드 쓰기: 동시 삽입으로 UPSTREAM 보다 처리량 1.70 배, Put p99 지연은 절반 이하다.

운영 관점의 선택지: KV Store 는 스키마가 없어 key 에 prefix(테넌트, 테이블, 종목 등)를 두는 경우가 많다. 예를 들어 MyRocks 의 모든 key 는 4 바이트 index id 로 시작한다. 이런 환경은 자연스럽게 HashSkipList 에 맞는 워크로드가 되며, `prefix_extractor` 만 설정하면 바로 쓸 수 있다.

upstream 기여 가능성: 변경은 RocksDB 의 기존 인터페이스 안에서 이루어졌고, 커밋마다 단위 테스트와 release note 를 갖췄다.

2. 팀 구성

항목	내용
팀명	**돌디비**
구성원	1인
이름	신승호
학과	소프트웨어학과
학번	32212326
역할	문제 정의, RocksDB 구조 설계와 구현, 실험 설계, 결과 분석

3. 개발 일정

기간	내용	결과
2025-04 ~ 2025-08	오픈소스SW빅데이터 과목: HashSkipList의 Bucket을 SkipList에서 InlineSkipList로 교체, 멀티 스레드 입력 구현(bucket 생성 동기화를 Double-Checked Locking으로 처리)	동시 쓰기 지원, 과목 발표
2026-10 ~ 2026-11	졸업작품: prefix_list_와 PrefixOrderedIterator 개발(복사 없는 total-order 순회), prefix_list_ 높이 정리, known_prefix 캐시. 기존 변경을 RocksDB 최신 main 위에 커밋 5개로 다시 정리(bucket 설치는 CAS)	브랜치 hash_skiplist_improvements
2026-11	벤치마크 웹 대시보드, 측정기, 실험, 문서	본 보고서의 측정 결과

4. AI 사용 범위

이 프로젝트는 생성형 AI 도구(Anthropic Claude, Claude Code)를 사용했다. 사용 범위는 다음과 같다.

영역	본인	AI
문제 정의, 목표 설정	과목 발표 이후의 문제 인식, 1차·2차 목표 설정	–
RocksDB HashSkipListRep 수정 (커밋 5개)	Bucket 교체 코드 작성 (DCL → CAS) InsertConcurrently() 코드 작성 Insert Key Encode 및 Allocate 로직 작성 PrefixOrderedIterator 작성	PrefixOrderedIterator 로직 오류 발견 - KV domain 유무 검증 및 처리 로직 개선 - comparator 범용성 개선 (Bytewise 대신 userComparator 사용)
실험 설계	비교할 조건과 질문 결정(높이, 순차·무작위 key, 점 조회, 정확도 검증 등), 결과 해석과 채택	측정 스크립트 작성과 실행, 수치 집계
벤치마크 웹 설계	요구사항과 구성 설계: 카드별 실험, 세 Memtable 동시 실행, 1초 지표 수집, 화면 구성, 애니메이션 장면 구성	–
벤치마크 웹 구현	결과 확인과 수정 요청	Spring 백엔드, 프론트엔드, 측정기의 지표·검증 기능, 애니메이션 페이지 구현
문서	검토와 수정	README와 본 보고서 초안 작성

5. 벤치마크 환경

5.1 하드웨어와 소프트웨어

항목	사양
CPU	Intel Core i9-12900K, 16코어 24스레드 (P코어 8개 = 논리 0-15, E코어 8개 = 16-23), 최대 5.2GHz
캐시	L2 14MB, L3 30MB
메모리	124GB
저장장치	Samsung PM9A1 NVMe 2TB × 2
OS	Rocky Linux 10.2, Linux 6.12
컴파일러	GCC 14.3.1 (C++20, -O3 -march=native -fno-rtti), CMake 3.31.8
RocksDB	facebook/rocksdb main e591f5f1a(2026-11-06) + 커밋 5개, Release 공유 라이브러리
웹 서버	OpenJDK 21.0.8, Spring Boot 3.5.9, Gradle 8.10
데이터	TPC-E EGen v1.14.0 Trade.txt (고객 5,000, scale factor 500, 초기 거래 300일, 28,664,881행, 3.2GB)

5.2 측정 방법

벤치마크 웹의 카드 4 종을 기본 설정으로 5 회씩 실행했다. 표의 값은 5 회 평균(± 표준편차)이고, 그래프의 막대는 평균, 오차 막대는 최소~최대 범위다.

한 번의 실행에서 세 Memtable 을 별도 프로세스로 동시에 돌리고, 서로 다른 P 코어 4 개씩(2-5, 6-9, 10-13)에 고정한다. 같은 KV 를 같은 순서로 넣는다.

공통 설정: WAL 끔, HashSkipList bucket 100 만 개, HEAD 와 UPSTREAM 의 bucket 최대 높이 12(SKIPLIST 기본값과 같음), prefix 는 key 의 첫 |까지(종목).

카드	KV	memtable	key	쓰기	측정
Flush 시간	10,000,000	64MB	trade_종목|t_id	큐, 소비자 1	flush 시간, 끝에 DB::Flush, 정확도 검증
멀티 스레드 쓰기	8,000,000	512MB	trade_종목|계좌|t_id	preload, 소비자 4, 동시 쓰기	처리량, Put 지연, 정확도 검증
Range query	2,000,000	512MB	trade_종목|t_id	preload, 소비자 1	prefix scan, total-order scan 각 8초, scan당 최대 100개
Point query	5,000,000	2048MB	trade_종목|계좌|t_id	preload, 소비자 1	Get 있는 key / 없는 key / 없는 종목 각 4초

6. 벤치마크 웹 서버 아키텍처



그림 1. 벤치마크 웹 서버 구성과 측정 한 번의 흐름

6.1 기술 스택

계층	기술	역할
프론트엔드	HTML5, CSS, Vanilla JavaScript(ES2020), SVG, uPlot 1.6.30	카드 4종, 설정 창, 1초 폴링으로 실시간 그래프, 통계·정확도 패널, 아키텍처 애니메이션(38단계)
웹 서버	Java 21, Spring Boot 3.5.9(Spring Web MVC, 내장 Tomcat), Gradle 8.10	REST API, 측정 프로세스 실행·관리, 1초 지표 수집, perf 카운터와 워크로드 분석
측정기	C++20, RocksDB 공유 라이브러리, POSIX 소켓	TPC-E 거래를 KV로 변환해 쓰기·scan·Get, Prometheus 형식 /metrics 노출, 정확도 검증
실행 환경	Linux taskset, perf stat	코어 고정, 하드웨어 카운터(사이클, IPC, 캐시·TLB 미스)

6.2 지표 수집: Prometheus 형식 pull

측정기는 HTTP 서버를 띄워 /metrics 에 Prometheus 텍스트 형식(# HELP, # TYPE, 이름{라벨} 값)으로 지표를 노출한다. 별도의 Prometheus 서버 없이 Spring 백엔드가 1 초마다 직접 요청해(pull) 파싱한다.

지표 값은 요청 시점에 계산한다: RocksDB 속성(DB::GetIntProperty 로 memtable 크기, flush·compaction 진행, L0 파일, SST 크기), 누적 통계(Statistics 의 ticker 와 histogram), flush 완료 이벤트(EventListener), 프로세스 CPU 와 RSS(/proc).

백엔드는 직전 값과의 차이로 초당 처리량, CPU 사용률, scan·Get 평균 지연을 계산해 1 초 간격 시계열을 만들고, 브라우저는 이를 GET /api/benchmarks/{id}/run 으로 받아 그린다.

측정이 끝나면 측정기가 표준 출력의 마지막 RESULT key=value ... 줄로 지연 분위수(p50, p99), scan·Get 결과, 검증 해시를 내고, 같은 지표를 파일(final.prom)로도 남긴다.

6.3 화면

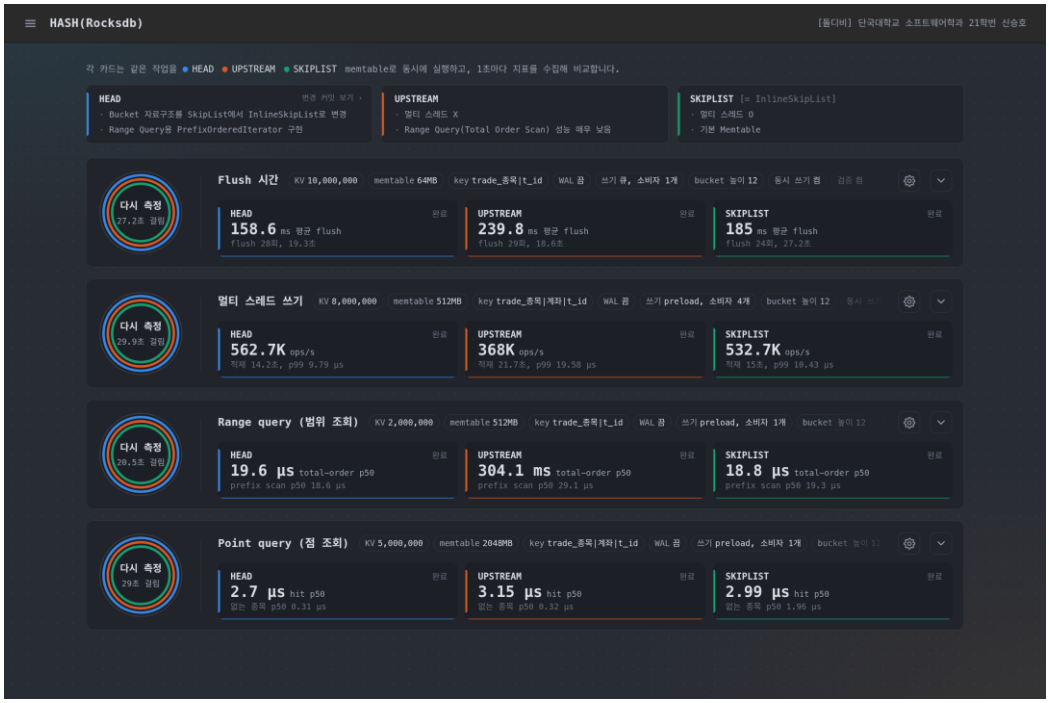


그림 2. 벤치마크 페이지: 카드마다 세 Memtable을 동시에 실행하고 결과를 나란히 비교



그림 3. 카드를 펼친 화면(Range query): 요약 비교, 1초 간격 시계열, 메모리·CPU, 하드웨어 카운터

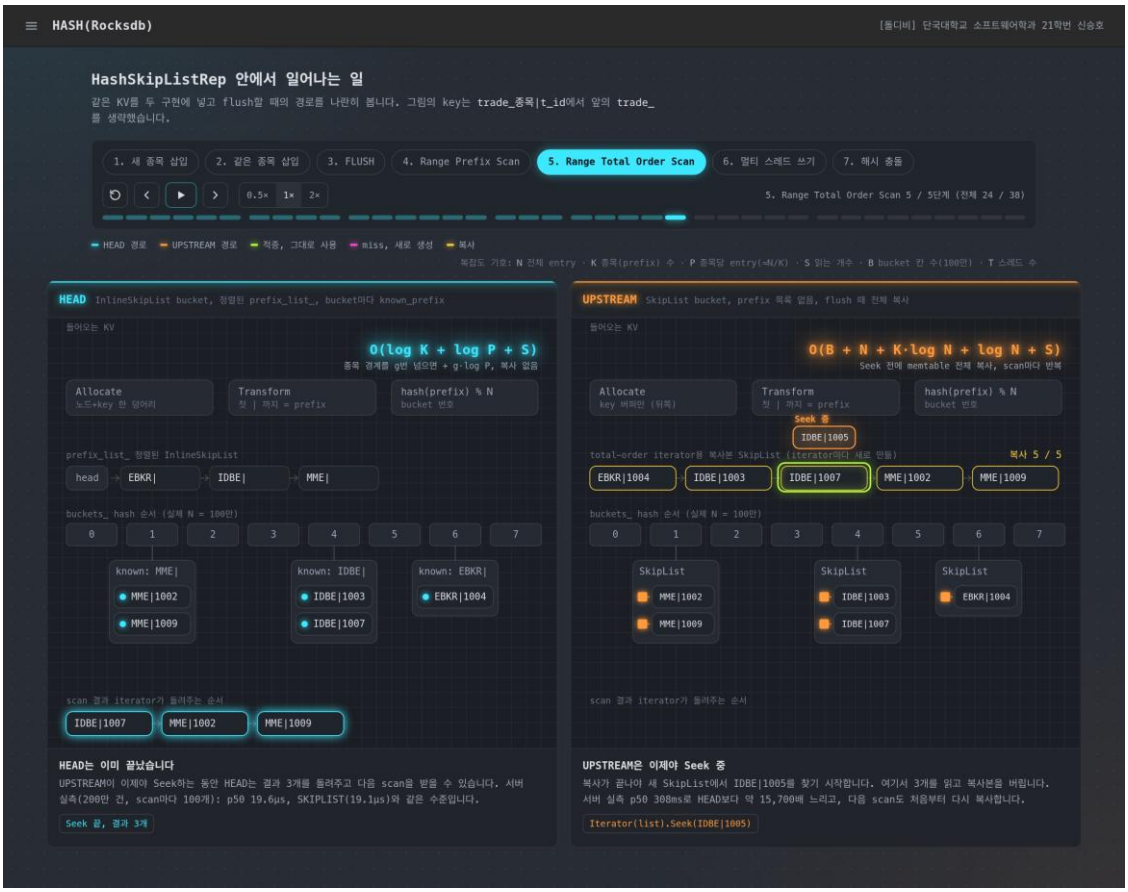


그림 4. 아키텍처 페이지: HEAD와 UPSTREAM의 동작을 같은 장면으로 나란히 애니메이션(Range Total Order Scan 장면)

7. 워크로드 설명

입력은 TPC-E의 거래 테이블(Trade.txt)이다. 한 행이 KV 하나가 되며, value는 종목을 뺀 나머지 열이다. key 형식은 두 가지이고, prefix는 둘 다 종목(첫 |까지)이다. 같은 데이터라도 key 형식에 따라 bucket 안에서의 삽입 패턴이 순차(sequential)와 무작위(random)로 나뉜다.

key 형식	예	bucket 안 삽입	쓰는 카드
trade_종목|t_id	trade_IDBE|200000000000003	순차: 항상 꼬리에 추가	Flush, Range query
trade_종목|계좌|t_id	trade_IDBE|43000004472|200000000000003	무작위: 계좌 순 정렬이라 중간에 끼어듦	멀티 스레드 쓰기, Point query

시간순 key 는 bucket 안에서 새 key 가 항상 가장 크므로, 직전 삽입 위치를 다시 쓰는 fast-path(InlineSkipList 의 seq_splice_, SkipList 의 prev_)로 탐색을 건너뛴다. 반면 SKIPLIST 는 전역 리스트 하나라 연속한 두 거래가 대개 다른 종목이고, 매번 처음부터 탐색한다. 그래서 같은 데이터가 Hash 계열에는 순차, SKIPLIST 에는 무작위 쓰기가 된다.

계좌 key 는 bucket 안이 계좌 번호 순이라 새 거래가 bucket 중간에 끼어든다. 매 삽입이 탐색이므로 노드와 key 가 붙어 있는 InlineSkipList 의 이점이 드러나고, 멀티 스레드에서는 삽입 위치가 흩어져 동시 삽입의 효과가 커진다.

규모: 종목 3,425 개, 계좌 5,000 개, 종목당 거래는 앞 200 만 행 기준 92~1,369 건(중앙값 569). 평균 key 26 바이트(계좌 key 38 바이트), value 111 바이트.



그림 5. key 형식에 따른 삽입 패턴

8. 실제 실험 결과

카드 4종을 5회씩 실행한 결과다. 표의 값은 평균 ± 표준편차, 괄호 안은 UPSTREAM 대비 변화다. 그래프의 오차 막대는 5회의 최소~최대 범위다.

8.1 Flush 시간

지표	HEAD	UPSTREAM	SKIPLIST
평균 flush 시간 (ms)	149.7 ± 5.2 (-32%)	220.0 ± 9.5	167.8 ± 8.2 (-24%)
flush 횟수	28 ± 0 (0%)	28 ± 0	24 ± 0 (-14%)
쓰기 처리량 (ops/s)	736,260 ± 21,384 (-1%)	746,216 ± 15,786	470,762 ± 14,531 (-37%)
Put p99 (µs)	2.48 ± 0.07 (+3%)	2.42 ± 0.06	3.85 ± 0.09 (+59%)
적재 시간 (초)	13.69 ± 0.39 (+1%)	13.50 ± 0.28	21.35 ± 0.64 (+58%)

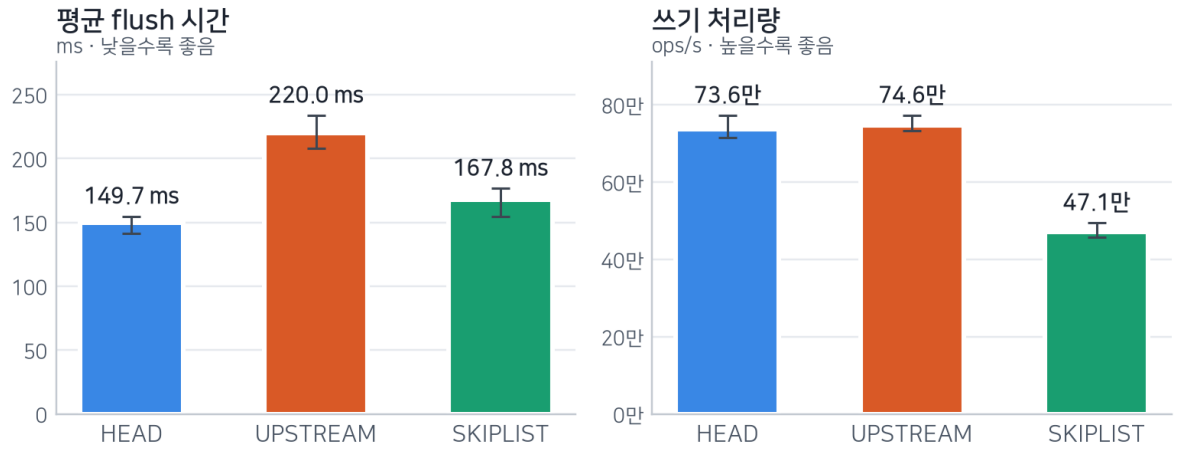


그림 6. Flush 카드 (5회 평균, 오차 막대 = 최소~최대)

HEAD의 flush는 UPSTREAM보다 **32% 짧다**. UPSTREAM은 flush가 쓰는 iterator를 열 때 memtable 전체(약 35만 건)를 새 SkipList로 복사한 뒤에야 SST를 쓰기 시작하지만, HEAD의 PrefixOrderedIterator는 복사 없이 prefix_list_ 순서로 bucket을 바로 읽는다. 쓰기 처리량은 HEAD와 UPSTREAM이 같고 SKIPLIST보다 56% 높다. 시간순 key가 Hash 계열에는 bucket 안 순차 삽입이기 때문이다(7장).

8.2 멀티 스레드 쓰기

지표	HEAD	UPSTREAM	SKIPLIST
쓰기 처리량 (ops/s)	624,979 ± 35,375 (+70%)	367,815 ± 2,077	484,219 ± 47,205 (+32%)
Put p50 (µs)	5.48 ± 0.07 (-46%)	10.08 ± 0.06	5.95 ± 0.11 (-41%)
Put p99 (µs)	9.56 ± 0.16 (-51%)	19.43 ± 0.29	10.59 ± 0.11 (-46%)
적재 시간 (초)	14.65 ± 0.78 (-38%)	23.59 ± 0.15	18.47 ± 1.59 (-22%)

지표	HEAD	UPSTREAM	SKIPLIST
평균 flush 시간 (ms)	1,216 ± 59 (-42%)	2,092 ± 78	1,185 ± 48 (-43%)

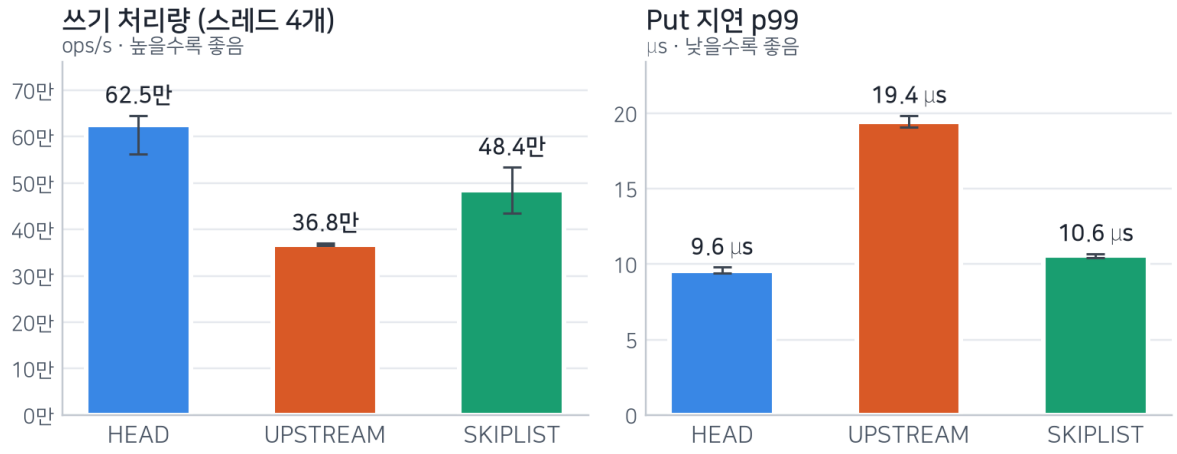


그림 7. 멀티 스레드 쓰기 카드 (스레드 4개)

스레드 4개가 동시에 쓸 때 HEAD의 처리량은 UPSTREAM의 **1.70배**이고, Put p99 지연은 51% 낮다. UPSTREAM은 동시 삽입을 지원하지 않아 RocksDB가 쓰기를 모아 리더 한 스레드가 모두 넣는다. HEAD는 SKIPLIST보다도 29% 높다. 삽입 위치가 종목별 bucket으로 나뉘어 스레드끼리의 경합과 탐색 길이가 줄기 때문이다.

8.3 Range query (범위 조회)

지표	HEAD	UPSTREAM	SKIPLIST
prefix scan p50 (µs)	18.56 ± 0.18 (-36%)	29.16 ± 0.14	19.33 ± 0.00 (-34%)
prefix scan p99 (µs)	23.58 ± 0.92 (-31%)	34.25 ± 0.46	24.65 ± 1.34 (-28%)
prefix scan 초당 횟수	56,639 ± 441 (+55%)	36,501 ± 115	54,103 ± 205 (+48%)
total-order scan p50 (µs)	19.63 ± 0.12 (1/15,575)	305,764.76 ± 2,297.31	18.87 ± 0.11 (1/16,204)
total-order scan p99 (µs)	24.40 ± 0.28 (1/13,262)	323,562.91 ± 13,796.79	23.37 ± 0.21 (1/13,844)
total-order scan 초당 횟수	50,296.1 ± 282.6 (×15,149)	3.3 ± 0.1	52,455.1 ± 315.5 (×15,800)

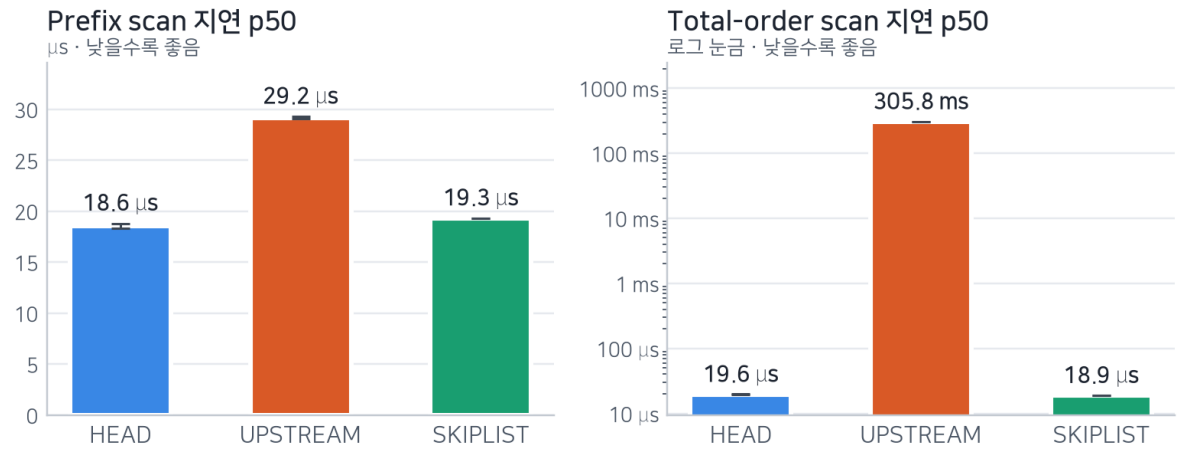


그림 8. Range query 카드 (오른쪽은 로그 눈금)

이 프로젝트의 1차 목표다. 종목을 넘는 total-order scan에서 UPSTREAM은 scan마다 memtable 200만 건을 복사해 p50이 **305.8 ms**이고 초당 3.3회밖에 처리하지 못한다. HEAD는 **19.63 µs**로 약 **15,575배** 빠르고, 기본 SkipList(18.87 µs)와 같은 수준이다. 한 종목 안의 prefix scan도 HEAD가 UPSTREAM보다 36% 빠른데, 이는 노드와 KV를 한 덩어리로 할당해 entry마다 읽는 메모리 위치가 하나로 줄었기 때문이다.

8.4 Point query (점 조회)

지표	HEAD	UPSTREAM	SKIPLIST
있는 key p50 (µs)	2.698 ± 0.014 (-15%)	3.165 ± 0.018	2.998 ± 0.014 (-5%)
있는 key p99 (µs)	5.613 ± 0.029 (-6%)	5.997 ± 0.029	6.163 ± 0.054 (+3%)
있는 key 초당 횟수	347,884 ± 1,300 (+16%)	300,731 ± 1,029	314,319 ± 1,125 (+5%)
있는 종목의 없는 key p50 (µs)	2.666 ± 0.014 (-13%)	3.069 ± 0.029	2.934 ± 0.042 (-4%)
없는 종목 p50 (µs)	0.319 ± 0.007 (-2%)	0.325 ± 0.009	2.045 ± 0.074 (+529%)

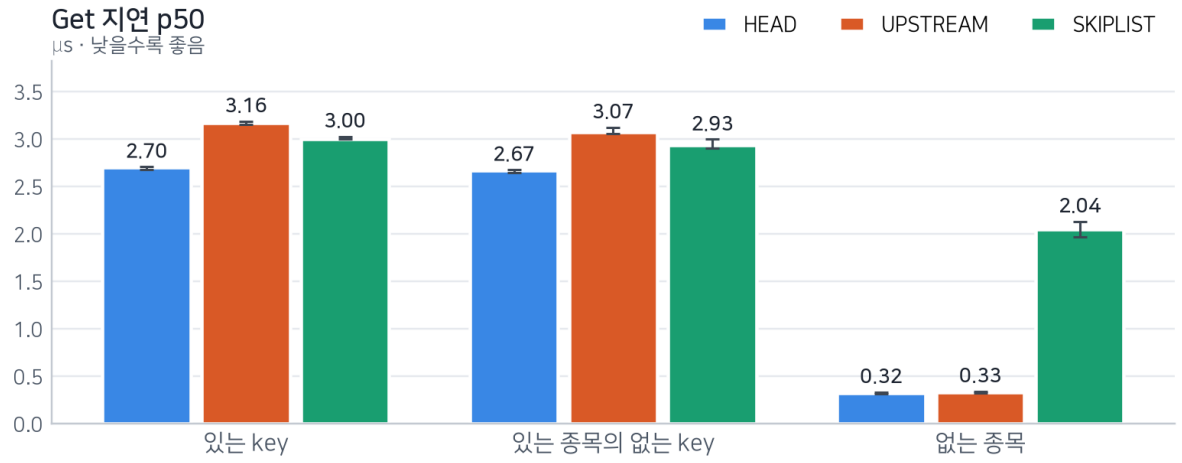


그림 9. Point query 카드 (500만 건, 2GB memtable)

있는 key 조회에서 HEAD는 UPSTREAM보다 15%, SKIPLIST보다 10% 빠르다. Get은 fast-path 없이 매번

bucket을 탐색하므로 노드와 key가 붙은 InlineSkipList의 캐시 이점이 그대로 드러난다. 한 번도 쓰지 않은 종목은 Hash 계열이 빈 bucket에서 바로 끝나 **0.32 µs**, SKIPLIST는 전체 리스트를 탐색해 2.04 µs로 약 6.4배 차이가 난다.

8.5 정확도 검증

성능이 빨라도 결과가 다르면 의미가 없다. 측정기의 --verify 옵션은 쓰기가 끝난 뒤 다음 두 단계를 수행한다.

1 단계(flush 전): memtable 과 SST 를 포함한 DB 전체를 key 순서로 읽어 KV 개수, key 가 엄격히 증가하는지, 모든 (key 길이, key, value 길이, value)의 64 비트 FNV-1a 해시를 낸다. HashSkipList 는 이때 PrefixOrderedIterator 로 순회한다.

2 단계(flush 후): 남은 memtable 을 모두 flush 한 뒤 SST 에서 같은 방법으로 다시 읽는다. flush 경로도 PrefixOrderedIterator 를 쓰므로 flush 결과까지 검증된다.

SST 파일 자체의 해시는 파일마다 들어가는 블록 checksum 과 속성(생성 시각 등) 때문에 원래 다르므로, 내용 해시로 비교한다.

카드	회차	KV 개수	내용 해시 (flush 전 = flush 후)	판정
Flush	1	10,000,000	75e19f8a94ce3bc9	세 Memtable 일치
Flush	2	10,000,000	75e19f8a94ce3bc9	세 Memtable 일치
Flush	3	10,000,000	75e19f8a94ce3bc9	세 Memtable 일치
Flush	4	10,000,000	75e19f8a94ce3bc9	세 Memtable 일치
Flush	5	10,000,000	75e19f8a94ce3bc9	세 Memtable 일치
멀티 스레드	1	8,000,000	7e2d76fb9a1bf3dc	세 Memtable 일치
멀티 스레드	2	8,000,000	7e2d76fb9a1bf3dc	세 Memtable 일치
멀티 스레드	3	8,000,000	7e2d76fb9a1bf3dc	세 Memtable 일치
멀티 스레드	4	8,000,000	7e2d76fb9a1bf3dc	세 Memtable 일치
멀티 스레드	5	8,000,000	7e2d76fb9a1bf3dc	세 Memtable 일치

두 카드의 5회 실행 모두에서 세 Memtable의 KV 개수, 정렬 여부, flush 전후 내용 해시가 **모두 일치**했다. HEAD의 새 iterator와 flush 경로가 기본 SkipList와 정확히 같은 데이터를 같은 순서로 내놓는다는 뜻이다.

8.6 목표 달성

목표	근거 (5회 평균)	판정
1차: Range Query 난제 해결	total-order scan p50 19.63 µs (UPSTREAM 305.8 ms, 약 15,575배), SKIPLIST 18.87 µs와 같은 수준. flush 32% 단축. 정확도 검증 전부 일치	달성
2차: 운영 가능한 성능	모든 카드에서 UPSTREAM 이상. 멀티 스레드 쓰기 1.70배, Put p99 51% 감소	달성
2차: SkipList보다 나은 경우	시간순 쓰기 처리량 +56%, flush 11% 단축, 멀티 스레드 쓰기 +29%, 있는 key Get 10% 단축, 없는 종목 Get 약 6.4배	달성

정리하면, HashSkipList를 기피하게 만든 세 문제(멀티 스레드, Range Query, 메모리 배치)를 모두 해결했고, Range Query는 기본 SkipList와 같은 수준까지 끌어올렸다. 동시에 prefix로 나뉘는 워크로드에서는 쓰기, flush, 점 조회 모두 기본 SkipList보다 빠르다. HashSkipList는 이제 prefix가 있는 운영 워크로드에서 충분히 고려할 만한 Memtable이다.

측정 한계: 세 Memtable을 동시에 실행하므로 코어는 나뉘지만 L3 캐시와 메모리 대역폭은 공유한다. 별도의 단독 실행 실험(experiments/)에서도 같은 경향을 확인했다. bucket 최대 높이는 12 기준이며, RocksDB 기본값 4에서는 큰 memtable에서 Hash 계열이 느려진다.